

WA3897-CAP Lesson Guide

FedEx AI Native FDE Capstone



Part of
Accenture

© 2026 Ascendient, LLC

Revision 1.0.0 published on 2026-05-29

No part of this book may be reproduced or used in any form or by any electronic, mechanical, or other means, currently available or developed in the future, including photocopying, recording, digital scanning, or sharing, or in any information storage or retrieval system, without permission in writing from the publisher.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

To obtain authorization for any such activities (e.g., reprint rights, translation rights), to customize this book, or for other sales inquiries, please contact:

Ascendient, LLC
1 California Street Suite 2900
San Francisco, CA 94111
<https://www.ascendientlearning.com>

USA: 1-877-517-6540, email: getinfousa@ascendientlearning.com
Canada: 1-877-812-8887 toll free, email: getinfo@ascendientlearning.com

Table of Contents

1. AI Agent Development Capstone — Introduction.....	4
1.1. Course Outline	5
1.2. What You Will Build.....	6
1.2.1. Track A — Procurement Intelligence Agent	6
1.2.2. Track B — Release Readiness Agent	6
1.3. Tools and Framework.....	6
1.4. RAPID DevSecOps Controls	7
1.5. Reflection	7
1.6. Additional Resources.....	7
2. AI Agent Development Capstone — Conclusion.....	9
2.1. Outline	10
2.2. What You Accomplished	10
2.3. Track-Specific Accomplishments	11
2.3.1. Track A — Procurement Intelligence Agent	11
2.3.2. Track B — Release Readiness Agent	11
2.4. Reflection	11
2.5. Additional Resources.....	12

AI Agent Development Capstone — Introduction

LESSON 1

-
- ✓ Describe the two capstone tracks and select the one your group will build
 - ✓ Explain how spec-driven development shapes the workflow for each session
 - ✓ Identify the tools — Pydantic AI, OpenSpec, and GitHub Copilot Agent Mode — you will use across all five sessions
 - ✓ State the RAPID DevSecOps controls your implementation must satisfy by Session 4

This course puts the skills from AIC-1102 into practice. Over five half-day sessions your group will design, build, test, and peer-review a working AI agent — one that reads real data, applies real business rules, and produces structured output a decision-maker can act on.

1.1. Course Outline

1**Track Selection**

Choose Track A (greenfield Procurement Agent) or Track B (brownfield Release Agent). Both tracks reach the same learning outcomes; the domain and starting conditions differ.

2**Session 1 — Domain Exploration and OpenSpec Proposals**

Read the mock data, explore the domain with OpenSpec, and draft specifications for all components your agent must build.

3**Session 2 — Models and Implementation Begins**

Implement your Pydantic data models, data loader, and first tool function. Verify each component against its OpenSpec spec before moving on.

4**Session 3 — Agent Wiring**

Connect all tools into a Pydantic AI agent with a system prompt that enforces correct decision priority. Run the agent manually against sample requests and verify the outputs.

5**Session 4 — Testing and RAPID Peer Review**

Write a pytest suite covering all required decision outcomes. Invoke the RAPID ITC.009 peer review Agent Skill and address every finding.

6**Session 5 — Integration, Go/No-Go, and Showcase**

Run the agent against all sample requests, complete the Go/No-Go checklist, and present your agent and design decisions to the group.

1.2. What You Will Build

1.2.1. Track A — Procurement Intelligence Agent

A greenfield advisory agent that pre-screens purchase requests for a FedEx procurement team. The agent performs four checks — budget, vendor duplication, policy compliance, and risk — and returns a structured recommendation (`approve` , `deny` , or `escalate`) with written rationale a procurement officer can act on directly.

1.2.2. Track B — Release Readiness Agent

A brownfield advisory agent that integrates alongside an existing CI/CD pipeline. The agent scans dependencies for CVEs, checks for blocking tickets, detects changelog gaps, and evaluates the CI gate result. It returns a structured recommendation (`release` , `hold` , or `escalate`) without modifying or replacing any existing pipeline behavior.

1.3. Tools and Framework

Tool	Role in this capstone
GitHub Copilot Agent Mode	Drives spec proposal, code generation, and peer review invocation throughout all sessions
OpenSpec CLI	Formalizes domain knowledge into machine-verifiable contracts before any code is written
Pydantic AI	Agent framework; <code>output_type=</code> constrains the LLM to return a typed, validated result
pytest / pytest-asyncio	Test framework for the required test suite (ITC.003 compliance artifact)

1.4. RAPID DevSecOps Controls

Your implementation must satisfy four RAPID controls by the end of Session 4:

Control	Requirement
ITC.014 — Commit Format	Every commit message must include a user story reference: <code>[US-NNN] message</code>
ITC.003 — Test Evidence	<code>pytest tests/ -v --tb=short --junitxml=docs/test-results.xml</code> must pass and the XML file must be committed
ITC.013 — Backout Plan	<code>backoutPlan.md</code> must have the stable baseline commit hash and contacts filled in before peer review
ITC.009 — Peer Review	The RAPID ITC.009 Agent Skill must be invoked in Session 4 and all findings rated and resolved or formally accepted in <code>docs/rapid-peer-review.md</code>

1.5. Reflection

Think about the following before Session 1 begins:

- What business rule violations do you think are hardest to catch automatically — and why?
- In a real deployment, who should have the final decision authority when an agent recommends escalation?
- How would you validate that the agent is not introducing new biases into the procurement or release process?

1.6. Additional Resources

- Pydantic AI documentation — <https://ai.pydantic.dev/>
- OpenSpec CLI reference — run `openspec --help` in your terminal
- RAPID DevSecOps Framework reference — `supporting/resources/rapid-criteria-reference.adoc`

- AIC-1102 copilot-instructions.md pattern reference — see your track's `copilot-instructions.md`

AI Agent Development Capstone — Conclusion

LESSON 2

-
- ✓ Summarize the design decisions your group made and the trade-offs each involved
 - ✓ Evaluate your agent's behavior against the expected outcomes in the sample data
 - ✓ Identify one improvement your group would make if you had an additional session
 - ✓ Connect the spec-driven, peer-reviewed workflow you practiced to production AI development

2.1. Outline

- **What You Accomplished** — a recap of the practices and artifacts you produced across the five sessions
- **Track-Specific Accomplishments** — what each group delivered, by track (Track A greenfield procurement, Track B brownfield release)
- **Reflection** — discussion prompts on the spec-first, prompt-precision, peer-review-findings, and production-delta moments
- **Additional Resources** — Pydantic AI, OpenSpec, RAPID framework, and Anthropic prompt engineering references

Over five sessions your group moved from an empty directory and a set of business requirements to a working, tested, peer-reviewed AI agent. This conclusion reflects on what you built, how you built it, and where the practices you applied show up in production teams.

2.2. What You Accomplished

- **Specified before implementing** — every component had an OpenSpec contract before a line of production code was written. The spec held the code accountable, not the other way around.
- **Constrained the LLM** — `output_type=` on the Pydantic AI agent means the model cannot return free text, a JSON blob, or a partial response. The structured output is validated at the framework level before your application code ever sees it.
- **Built testable, data-driven logic** — your tools read from `data/loader.py`, not directly from files. This made them mockable, testable, and easy to swap when the data source changes.
- **Satisfied RAPID controls** — `docs/test-results.xml`, `docs/rapid-peer-review.md`, and a completed `backoutPlan.md` are the same artifacts a FedEx production release requires before a change goes live.
- **Peer-reviewed with a tool, not just a teammate** — the RAPID ITC.009 Agent Skill evaluated your implementation against six criteria that a human reviewer might miss or apply inconsistently.

2.3. Track-Specific Accomplishments

2.3.1. Track A — Procurement Intelligence Agent

You built a greenfield advisory system from scratch. The agent performs four independent checks — budget, vendor duplication, policy compliance, and risk — and synthesizes them into a single, explainable recommendation. Every one of the 15 sample requests has a deterministic expected outcome your agent can now reach.

2.3.2. Track B — Release Readiness Agent

You integrated an advisory layer into an existing CI/CD pipeline without modifying a single brownfield module. The three functional pipeline modules (`runner.py` , `deploy.py` , `changelog_reader.py`) run exactly as they did before your integration. The agent adds CVE awareness, ticket blocking detection, and changelog coherence checking as a read-only advisory layer.

2.4. Reflection

Consider these questions individually, then discuss as a group:

- **Spec first.** At which point in the capstone did writing the spec before the code save your group the most time? Was there a moment when the spec caught a design mistake before it became a bug?
- **Prompt precision.** What was the most important change your group made to the system prompt? What decision did it fix, and why was the original language ambiguous?
- **Peer review findings.** Which peer review criterion produced the most useful findings for your group? Would you have caught those issues in a manual code review?
- **Production delta.** If this agent were deployed as a production service, what is the single biggest change you would make to the architecture?

2.5. Additional Resources

- Pydantic AI — structured output and tool use: <https://ai.pydantic.dev/tools/>
- RAPID DevSecOps Framework — full criteria reference: `supporting/resources/rapid-criteria-reference.adoc`
- OpenSpec CLI — advanced usage (`openspec verify-all` , spec inheritance):
run `openspec --help`
- Anthropic prompt engineering guide: <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>
- FedEx AI governance contacts — ask your instructor for the internal reference link